

EXPERIMENT1 REPRORT_

R . Vikas Raj

EE19B108 , Batch 1

Objectives :

- To learn the working of serial-parallel multiplier and booth's multiplier.
- Implementing both the above in a FPGA.
- Comparing algorithms by efficiency in terms of number of computations involved

Hardware and Software Used :

- EDGE Artix 7 board and a USB interfacing cable
- Xilinx Vivado 2022 IMPLEMENTATION OF BOOTH'S MUL

Implementation of Serial-parallel Multiplier

Code for serial parallel multiplier :

```
module mult_4x4(  
    input reset,clk,  
    input [3:0] A,B,  
    output reg [7:0] O, // 8-bits output
```

output wire Finish);

```
reg [3:0] State; // state machine
reg [8:0] ACC; // Accumulator
// logic to create 2 phase clocking when starting
assign Finish = (State==9)? 1'b1:1'b0; // Finish Flag
always@(posedge clk)
if(reset)
begin
State <= 0;
ACC <= 0;
O <= 0;
end
else if (State==0)
begin
ACC[8:4] <= 5'b00000; // begin cycle
ACC[3:0] <= A; // Load A (one of our inputs)
State <= 1;
end
else if (State==1 || State == 3 || State ==5 || State == 7)
// add/shift State
begin
if(ACC[0] == 1'b1)
begin // add multiplicand
ACC[8:4] <= {1'b0,ACC[7:4]} + B;
State <= State + 1;
end
else
begin
ACC <= {1'b0,ACC[8:1]} ; // shift right
State <= State + 2;
end
end
```

```

else if(State==2 || State == 4 || State ==6 || State ==8)
// shift State
begin
ACC <= {1'b0,ACC[8:1]}; // shift right
State <= State + 1;
end
else if(State == 9 )
begin
State <= 0;
O <= ACC[7:0]; // loading data of accumulator in output
end
endmodule

```

Code for Test Bench :

```

`timescale 1ns/1ps
module test();

reg reset,clk;
reg [3:0] A,B;

wire [7:0] O;
wire Finish;

mult_4x4 dut(reset, clk,A,B,O,Finish);
always begin
#5 clk = ~clk;
end

initial begin
clk = 0;

```

```

reset = 1;
A = 0; B = 0;
#10 reset = 1;
#10 reset = 0;
// Test case 1: A = 11 (0b1011), B = 12 (0b1100)
A = 4'b1011;
B = 4'b1100;
#120;

// Test case 2: A = 3 (0b0011), B = 3 (0b0011)
reset = 1;
#10 reset = 0;
A = 4'b0011;
B = 4'b0011;
#120;

#10 $finish;
end
endmodule

```

EXPLANATION

- The code for Serial parallel multiplier takes two 4 bit unsigned numbers and multiply it and store it in the Accumulator.
- Here first the accumulator is initialized with 9 bit number with first 4 bits(ACC[3:0]) loaded with Multiplier and remaining bits loads with zeros.
- This code first checks the lsb of multiplier if it is 0 then accumulator will be shifted right. If it is 1 then multiplicand is added to the ACC[7:4] and Accumulator is shifted towards right.
- To achieve this the code uses a state machine with 8 states where the odd states correspond to the operation of Adding and the even states correspond to the operation of shifting.

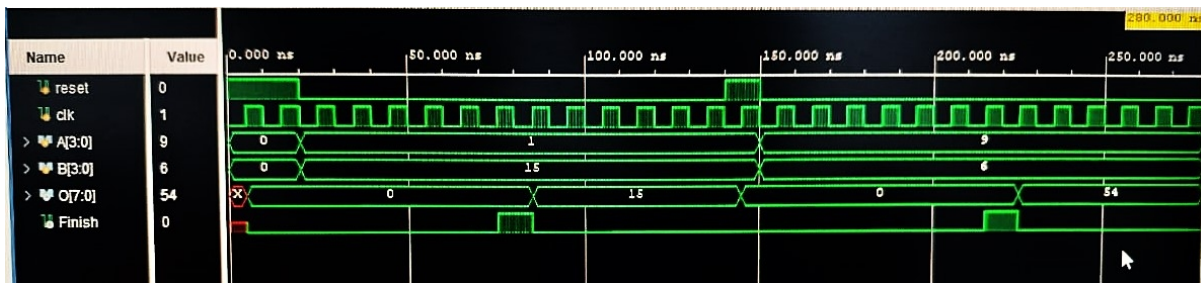


Figure 1: Timing Diagram For Serial Parallel

IMPLEMENTATION OF BOOTH'S MULTIPLIER:

Code for Booth's Multiplier:

```

module multiplier(prod, busy, mc, mp, clk, start);
//mc is the multiplicand & mp is the multiplier
output [7:0] prod; // output 8 bits
output busy;
input [3:0] mc, mp; // input 4 bits
input clk, start;
reg [3:0] A, Q, M; // all registers are of 4 bits
reg Q_1;
reg [2:0] count;
wire [3:0] sum, difference;
always @(posedge clk)
begin
if (start)
begin
A <= 4'b00;
M <= mc;
Q <= mp;
Q_1 <= 1'b0; // bit written to the left of lsb of number to be multiplied
count <= 3'b0;
end
else if (busy)
begin

```

```

case ({Q[0], Q_1})
2'b01 : {A, Q, Q_1} <= {sum[3], sum, Q};
2'b10 : {A, Q, Q_1} <= {difference[3], difference, Q};
default: {A, Q, Q_1} <= {A[3], A, Q};
endcase
count <= count + 1'b1;
end
end
alu adder(sum, A, M, 1'b0); // adder
alu subtractor(difference, A, ~M, 1'b1); //subtractor using 2's compliment
assign prod = {A, Q}; // make it fill up the arguments
assign busy = (count < 4);
endmodule // The following is an alu.It is an adder, but capable of subtraction:
// Recall that subtraction means adding the two's complement--  $a - b = a + (-b)$ 
//  $= a + (\text{inverted } b + 1)$ 
// The 1 will be coming in as cin (carry-in)
module alu(out, a, b, cin);
output [3:0] out;
input [3:0] a;
input [3:0] b;
input cin;
assign out = a + b + cin;
endmodule

```

Code For TestBench :

```

`timescale 1ns/1ps
module test_1();
// signals
reg [3:0] mc, mp;
reg clk, start;
// Outputs

```

```
wire [7:0] prod;
wire busy;
// device under test
multiplier dut(prod, busy, mc, mp, clk, start);
always begin
#10
clk = ~clk;
end
initial begin
clk = 0;
start = 0;
mc = 4'b0000;
mp = 4'b0000;
#10;
mc = 4'b0011;
mp = 4'b0101;
start = 1;
#10 start = 0;
wait(busy == 0);
#100;
start = 1;
mc = 4'b0111;// 7 in decimal
mp = 4'b0010;// 2 in decimal
#10
start = 0;
wait(busy == 0);
#100;
start = 1;
mc = 4'b0101; // 5 in decimal
mp = 4'b0010; // 2 in decimal
#10
start = 0;
wait(busy == 0);
```

```
#100 $finish;
end
endmodule
```

EXPLANATION :

- This code for booth multiplier takes two signed 4 bit number and multiplies it and assigns the output to the prod.
- This code checks the last two bits of the multiplier and do the necessary operations as per code.
- When Start is pressed, the inputs get loaded and the count is reset to 0.
- The correct value of product will be obtained when busy=0.

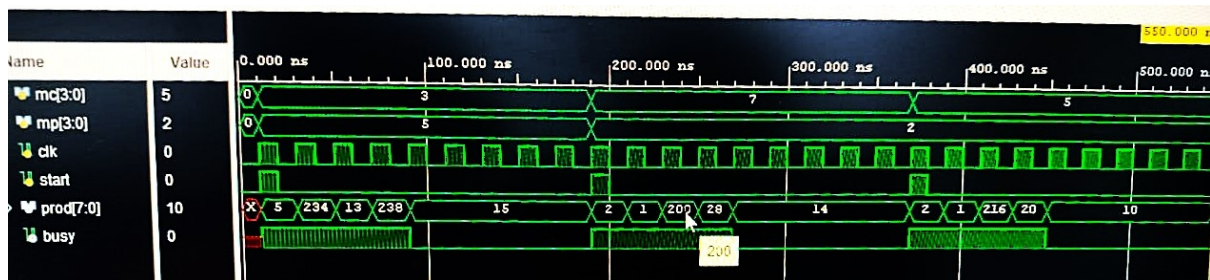
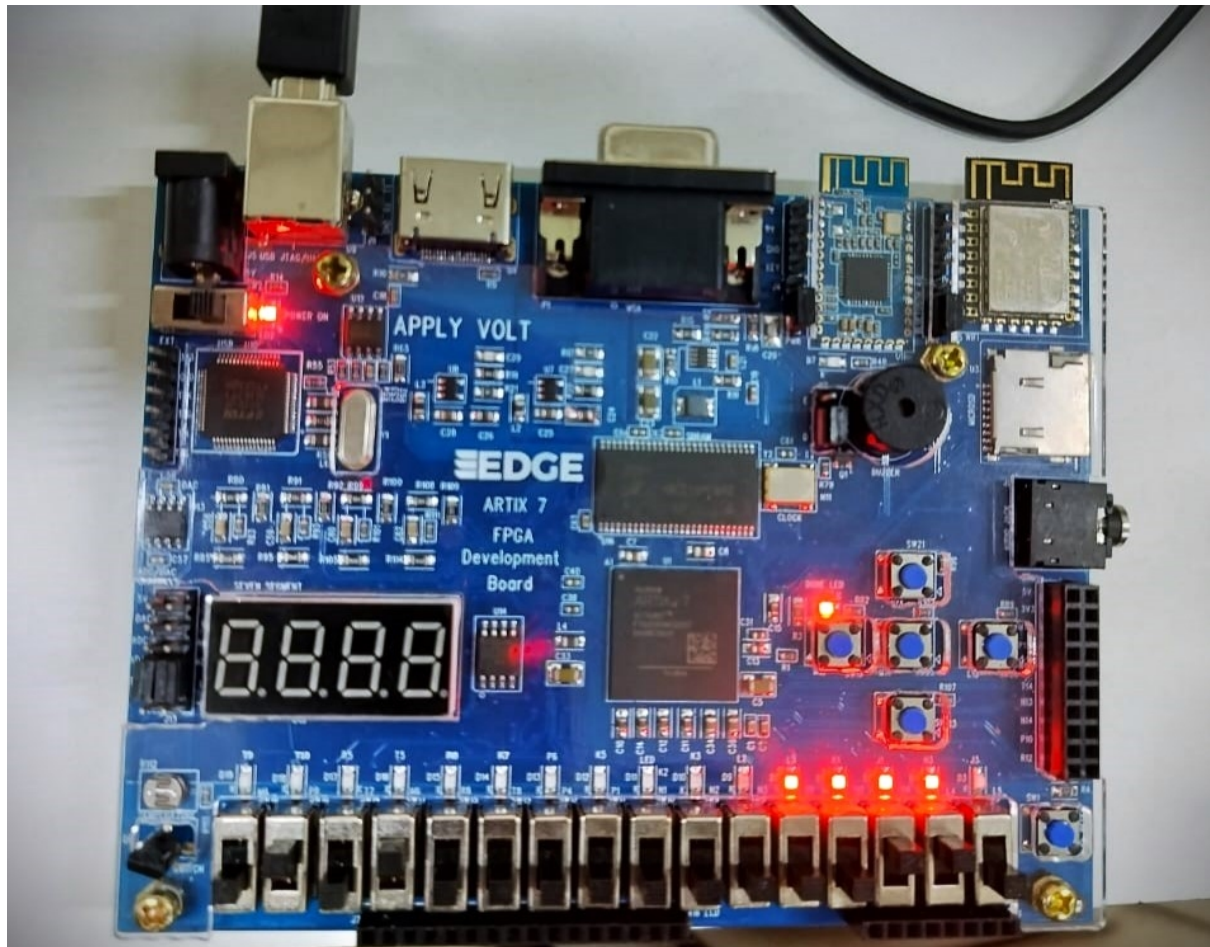


Figure 2: Timing Diagram for Booths Multiplier

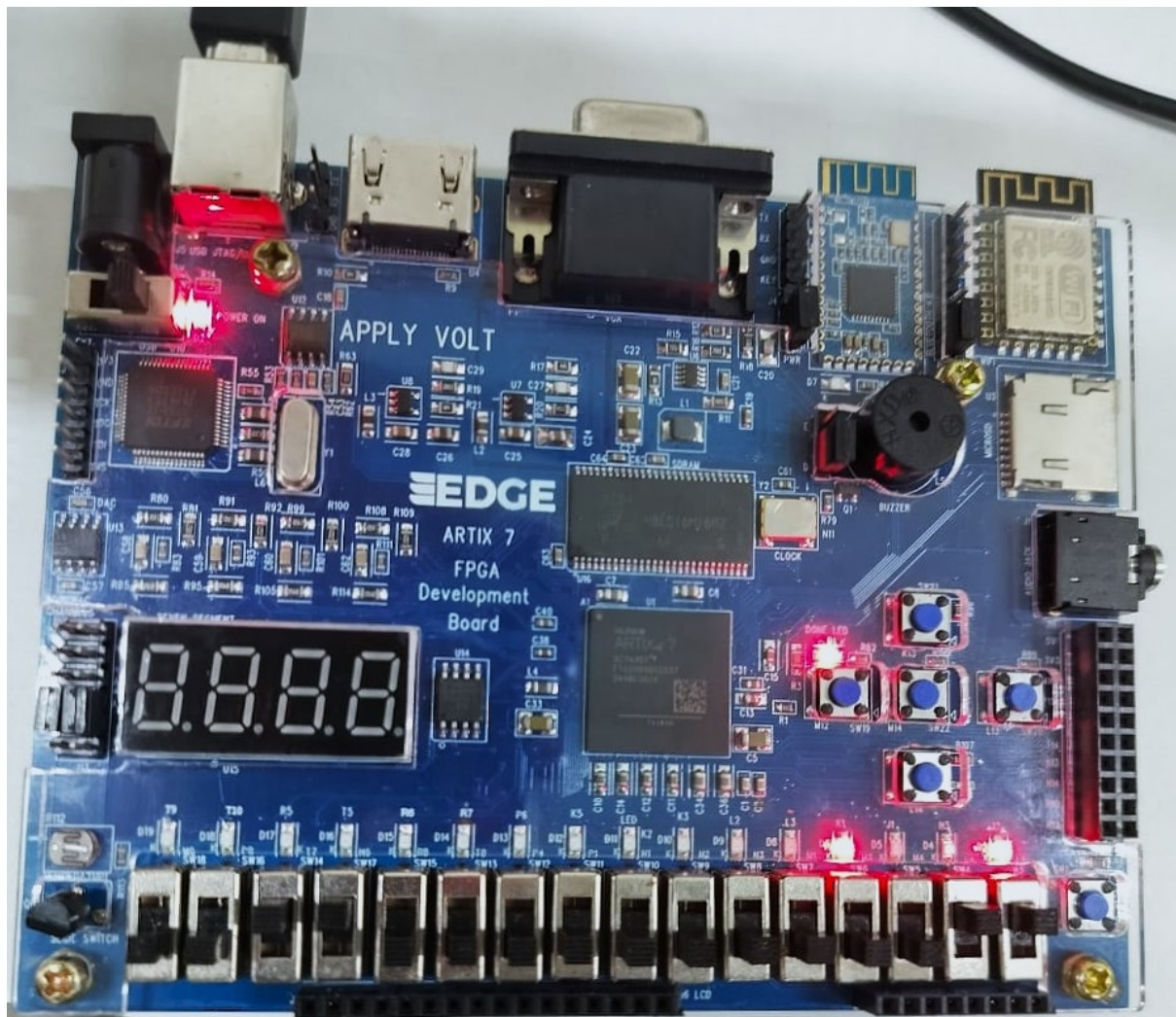
FGPA BOARD IMPLEMENTATION :

- To program a FPGA board first we should assign the input,output,reset keys in constraint files.
- In our Experiment we have implemented input has Slide switches,outputs is shown by LED's and reset is given by push buttons. .
- After finishing the constraints file set the file(file that should be implemented) at the top.
- Click on Generate bitstream.this will involve running synthesis and Implementation before finally generating the bitstream.

- Connect the FBGA through a interface cable to the PC running Xilinx and open the hardware manager once the BitStream is generated.
- Now by changing the inputs and reset button in FGPA board we can observe the output.



Serial-Parallel Multiplier output



Booths Multiplier output

INFERENCES AND CONCLUSION

From the timing diagrams provided above, we can see that the Serial Parallel multiplier takes 8 clock cycles to complete, while the Booths multiplier requires only 4 cycles. However, with the optimization of skipping states when multiplying by 0, the Serial Parallel multiplier's cycle count can occasionally be less than 8. Additionally, the Serial Parallel multiplier is designed for unsigned numbers, whereas the Booths multiplier handles signed numbers.

Overall, the Booth multiplier appears to be more efficient, as it involves fewer computations compared to the Serial-Parallel multiplier.